

Spectra — scheda progetto per il colloquio

Samuele Pio Provenzano · agente AI per la chimica, integrato in BioSpecInfo Repository:
`samupropio1-ship-it/BioSpecInfo-v11` · demo pubblica su GitHub Pages

In una frase

Un **agente conversazionale** che risolve problemi di chimica e fisica invocando 31 strumenti deterministici invece di rispondere a memoria, con ragionamento visibile e verificabile, funzionante su 8 configurazioni di modello di 4 fornitori diversi — senza alcun backend.

Numeri

Componente	3.935 righe, zero dipendenze runtime
Strumenti	31, su 13 aree scientifiche
Fornitori supportati	4 (Anthropic, Groq, Google, OpenRouter/xAI) — 3 configurazioni gratuite
Dataset interni esposti	9, oltre 800 record
Copertura di test	14 pagine, 84 sezioni, 18 tab — nessun errore JS

Le tre domande che mi aspetto, e le risposte

«Perché è un agente e non un chatbot?»

Perché il modello **non produce direttamente i numeri**. Quando gli si chiede il pH di un acido debole, invoca un risolutore che risolve l'equazione di secondo grado esatta e restituisce 2,875; il modello commenta il risultato, non lo inventa. Il ciclo di controllo concatena fino a 10 chiamate, e il prompt di sistema impone di verificare l'ordine di grandezza prima di rispondere.

La differenza è verificabile: ogni risultato numerico è accompagnato dalla traccia dello strumento che l'ha prodotto, conservata insieme alla risposta.

«Qual è la decisione tecnica di cui vai più fiero?»

Non aver usato `eval()` per il motore di calcolo.

Serviva un valutatore di espressioni matematiche arbitrarie. `eval()` avrebbe risolto il problema in tre righe. Ma l'agente legge contenuti esterni non fidati — risultati PubChem, abstract PubMed, pagine web — e un'iniezione in quei contenuti avrebbe potuto far eseguire codice arbitrario nel contesto della pagina, **dove risiede la chiave API dell'utente**.

Ho scritto un parser a discesa ricorsiva con un insieme chiuso di 27 funzioni. Poi l'ho attaccato: 10 tentativi di evasione (`window.localStorage` ,

`constructor` , `this` , `fetch(...)`), tutti respinti, verificato anche in un

browser reale tentando di leggere davvero la chiave.

È il tipo di scelta che costa mezza giornata e non si vede mai — finché non serve.

«Come hai verificato che funzioni?»

Su tre livelli.

Numerico: ogni risolutore confrontato con valori di riferimento da letteratura — pH 2,875 per l'acido acetico 0,1 M; 55,3 kJ/mol per l'energia di attivazione da due costanti cinetiche; 656,1 nm per la riga H- α ; 8,79 MeV/nucleone per il ^{56}Fe ; t critico 2,7764 per $df = 4$.

Robustezza: casi che *devono* fallire. Equazioni non bilanciabili, formule con parentesi sbilanciate, specie assenti dall'equazione, tentativi di iniezione. Un risolutore che non sa dire "non lo so" è peggio di uno assente.

Integrazione: Chromium headless su tutte le 14 pagine, 84 sezioni percorse una per una, e un ciclo completo simulato con ricerca web, sospensione e ripresa del turno.

Tre bug che ho trovato testando (e cosa insegnano)

1. Il parser di formule sbagliava $\text{Ca}_3(\text{PO}_4)_2$ — dava 215 invece di 310. La prima implementazione gestiva le parentesi con una pila di moltiplicatori scalari, che non regge i gruppi annidati. Riscritta con una pila di mappe di conteggi: ora regge anche $\text{K}_4[\text{Fe}(\text{CN})_6]$. *Lezione: i casi facili passano sempre. Servono i casi difficili scelti apposta.*

2. Dieci dichiarazioni CSS `calc()` **silenziosamente invalide.** In CSS gli operatori `+` e `-` richiedono spazi su entrambi i lati:

`calc(env(safe-area-inset-bottom,0px)+24px)` viene **scartato dal browser**.

Gli elementi cadevano nella posizione statica — ed era il motivo per cui il progetto conteneva due watchdog JavaScript che ne forzavano la posizione ogni due secondi. *Lezione: quando trovi una pezza, cerca la ferita.*

3. Il ragionamento del modello veniva catturato ma non mostrato. Il parser dello stream restituiva solo i `text_delta`: con un modello che ragiona, l'utente vedeva una lunga pausa e pensava fosse bloccato. Peggio, il pannello che avevo aggiunto spariva a fine turno, perché la chat viene ricostruita dai messaggi salvati. *Lezione: una funzionalità non è finita quando funziona, ma quando sopravvive al ciclo di vita della UI.*

Competenze dimostrate

- **Integrazione LLM in produzione:** function calling multi-fornitore, parsing di stream SSE, gestione dei blocchi di ragionamento con firma, ripresa di turni sospesi lato server, configurazione per-modello dei parametri API.
- **Sicurezza applicativa:** modello di minaccia dell'iniezione via contenuti esterni, superficie di esecuzione ridotta al minimo, gestione della quota di storage come vettore di guasto silenzioso.
- **Metodo di verifica:** casi di riferimento da letteratura, casi negativi

espliciti, test d'integrazione in browser reale automatizzato.

- **Dominio scientifico:** 13 aree fra chimica, chimica fisica, biochimica, farmacologia, fisica nucleare e astrofisica, implementate come risolutori esatti e validate.

| Limiti che dichiaro apertamente

La chiave API risiede nel browser: senza backend non c'è alternativa, e per un uso condiviso servirebbe un proxy con quota. I risolutori usano modelli semplificati dove la letteratura ne ha di più raffinati, e ogni strumento dichiara il metodo che ha usato. La verifica è funzionale e numerica, non formale: non c'è prova di correttezza, c'è un insieme di casi di riferimento.

*Documentazione completa: `docs/05-AI-Agent-Architecture.md` (IT) ·

`docs/en/05-AI-Agent-Architecture.md` (EN)*